

Ros2 Day2 hw1 AI 사용 보고서

휴머노이드 수습단원 이주원

목차

1. ai사용 부분
2. 대화내용
3. 깨달은 점

1. ai 사용 부분

이 과제에서 ai가 사용된 부분은 첫 번째로 퍼블리셔의 구조다. 처음에 생성자 안에서 입력을 받고 바로 발행하도록 짰는데 동작이 한 번만 되고 끝나 이유를 물어보았다. 같이 나 있던 컴파일 에러와 조건 분기 문제도 이때 함께 확인했다.

두 번째는 헤더 파일이다. 벡터를 쓰는데 헤더를 따로 포함하지 않아도 빌드가 되길래, 이대로 두어도 되는지 확인을 요청했다.

세 번째는 구독자의 출력 부분이다. 받은 배열을 어떻게 순회해서 출력해야 하는지 몰라서 물어보았다.

2. 대화 내용

1. `break;` — 컴파일 에러

`break` 는 반복문이나 `switch` 안에서만 쓸 수 있어. 여기는 함수의 `else if` 블록이라 짝이 될 루프가 없어서 "break statement not within loop or switch" 에러 나. 함수에서 빠져나가려면 `return;` 이야.

그리고 `num == -1` 은 도달 못 해. 위에서 `num >= 0` 이 걸러지고 -1은 음수니까 `else if`로 오긴 하는데... 이 건 맞네. 근데 -1이든 -5든 어차피 입력 없이 끝나는 건 같아서 굳이 나눌 필요는 없어. 남겨두려면 `return;` 으로 바꾸면 돼.

2. 생성자에서 입력받으면 안 됨

지금 `input_data()`, `publish_data()` 가 생성자 안에 있어. 그러면 노드 만들어질 때 딱 한 번만 입력받고, 그 뒤엔 `spin()` 이 아무것도 안 하면서 무한정 돌아. 게다가 생성자가 끝나기 전에 `publish()` 를 부르는 거라 아직 노드 초기화가 덜 된 상태에서 발행하는 셈이야. 구독자가 못 받을 수도 있어.

아까 말한 대로 main에서 돌려야 해. 이 노드는 구독도 타이머도 없으니 `spin()` 이 필요 없어:

```
cpp
int main(int argc, char **argv)
{
    rclcpp::init(argc, argv);
    auto node = std::make_shared<MyPublisher>();
    while (rclcpp::ok())
    {
        ↓
    }
```

+ 메시지를 입력하세요...

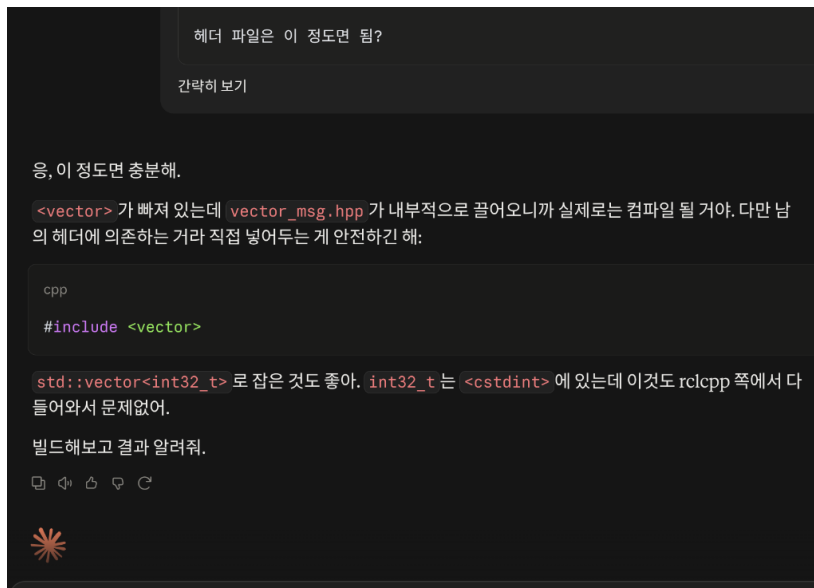
🔊 ▼ 🗨️

Claude는 AI이며 실수를 할 수 있습니다. 응답을 다시 한번 확인해 주세요.

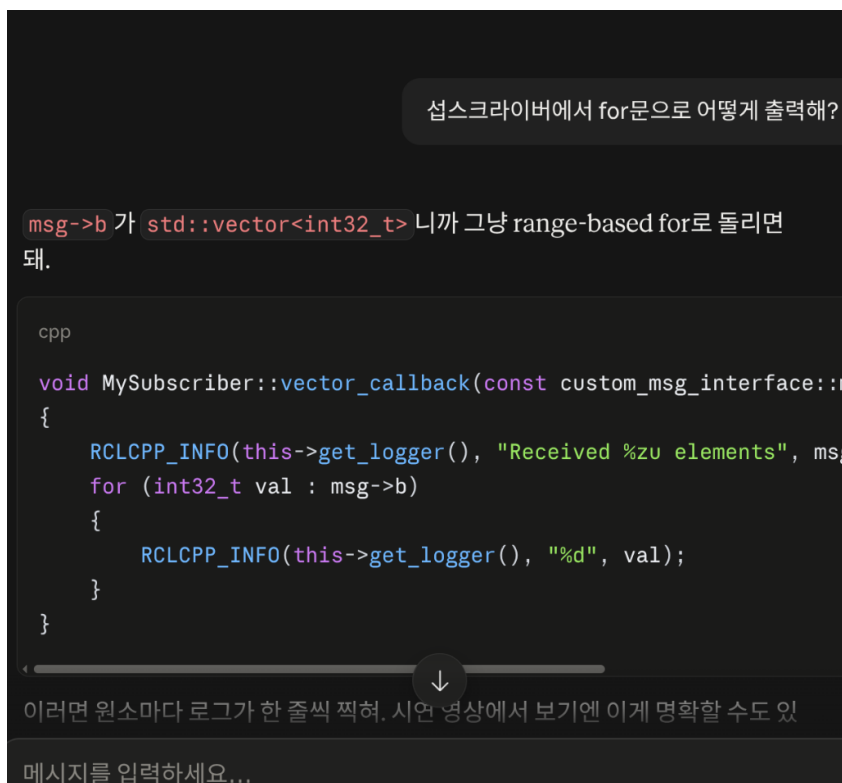
Opus 5 높음

처음에 퍼블리셔를 생성자 안에서 입력받고 바로 발행하도록 짰는데, 그렇게 하면 노드가 만들어질 때 한 번만 실행되고 그 뒤로는 아무 일도 일어나지 않는다는 답을 받았다. 게다가 생성자가 끝나기 전에 발행하는 것이라 노드 초기화가 덜 된 상태에서 보내는 셈이 되어 구독자가 받지 못할 수도 있다고 한다. 입력과 발행은 메인에서 반복문으로 돌려야 하며, 이 노드에는 구독도 타이머도 없으므로 노드를 계속 돌려주는 부분은 필요 없다는 설명도 함께 들었다.

반복문이 아닌 조건문 블록 안에 `break`를 써서 에러가 났던 것이고, 함수에서 빠져나가려면 `return`을 써야 한다는 답을 받았다. 또 음수를 -1과 그 외로 나누어 처리하려 했는데 앞의 조건에 이미 걸려서 뒤쪽 조건에는 도달하지 못한다는 지적을 받았고, 어차피 입력을 받지 않고 끝나는 것은 같으므로 굳이 나눌 필요가 없다는 답을 받았다.



벡터 헤더를 따로 포함하지 않아도 메시지 헤더가 내부적으로 끌어오기 때문에 컴파일은 되지만, 남의 헤더에 의존하는 것이라 직접 넣어두는 편이 안전하다는 답을 받았다. 배열 필드를 받을 자료형을 잡은 방식도 맞다는 확인을 받았다.



구독자에서 받은 배열을 출력하는 방법이다. 배열 필드가 C++에서는 벡터로 매핑되기 때문에 반복문으로 그대로 순회하면 되고, 원소마다 로그를 한 줄씩 찍으면 영상에서 확인하기에도 명확하다는 답을 받았다.

3. 깨달은 점

생성자는 노드를 만들 때 한 번만 실행되는 자리라는 것을 알게 되었다. 계속 반복해야 하는 동작을 여기에 넣으면 안 되고, 생성자는 발행자나 구독자를 등록하는 정도로만 써야 한다는 점을 알았다.

노드를 계속 돌려주는 부분이 항상 필요한 것은 아니라는 것을 알게 되었다. 콜백이나 타이머가 있는 노드에만 필요하고, 이번 퍼블리셔처럼 입력을 직접 기다리는 노드는 반복문으로 돌리는 것이 맞았다.

조건을 나누어 쓸 때 앞의 조건에 이미 포함되어 뒤쪽이 실행되지 않을 수 있다는 것을 알게 되었다. 코드를 늘리기 전에 그 분기가 실제로 도달 가능한지부터 확인해야 한다는 점을 깨달았다.

헤더가 다른 헤더를 통해 간접적으로 들어와 빌드가 되더라도, 실제로 쓰는 것은 직접 포함해두어야 안전하다는 것을 알게 되었다.

메시지에 정의한 가변 길이 배열이 C++에서는 벡터로 그대로 매핑된다는 것을 확인하였다. 덕분에 받은 데이터를 평소 쓰던 방식대로 다룰 수 있었다.